PARTE VI

Entorno de Software y Laboratorios

Herramientas computacionales en Python, modelado matemático y librerías científicas

Guía del entorno de experimentación computacional basado en el ecosistema científico de Python (SymPy, NumPy, SciPy, CVXPY y NetworkX). Presenta la estructura de los laboratorios prácticos, los cuadernos Jupyter interactivos y las librerías implementadas para la resolución computacional de los modelos de optimización y redes.

Víctor Aceña Gil · Antonio Alonso Ayuso

CC BY-SA 4.0 Internacional · Curso 2026-2027

Software utilizado en la asignatura

Los materiales prácticos de esta asignatura consisten en una serie de laboratorios desarrollados en Python y presentados en formato de cuaderno interactivo (`.qmd` de Quarto). Estos laboratorios son fundamentales para aplicar los conceptos teóricos vistos en clase, permitiendo al estudiante no solo ejecutar el código, sino también entender el razonamiento detrás de cada paso del análisis.

El código fuente completo de los laboratorios se encuentra disponible en el repositorio de la asignatura para garantizar el acceso abierto y la reproducibilidad de los resultados.

A continuación, se detalla el contenido de cada uno de los laboratorios incluidos en este repositorio.

Laboratorio 1: Introducción al Modelado y Cálculo Simbólico con SymPy

- **Descripción:** Este laboratorio introduce al estudiante en el uso del cálculo simbólico en Python mediante la biblioteca `SymPy` para obtener gradientes y matrices hessianas analíticas de forma exacta. Asimismo, se enseña el uso de `Autograd` para diferenciación automática numérica de funciones definidas por código. Por último, se aborda el estudio de la curvatura y la convexidad local y global en diversas dimensiones mediante el cálculo numérico de autovalores con `NumPy`.
- **Fichero Fuente:** `lab1_introduccion_sympy.qmd`

Laboratorio 2: Optimización No Lineal con Python

- **Descripción:** Esta sesión práctica se centra en la implementación y resolución de algoritmos de optimización continua sin y con restricciones. El alumno implementa manualmente el bucle del algoritmo de descenso de gradiente (búsqueda local de primer orden) para analizar su tasa de convergencia lineal. Posteriormente, se enseña a utilizar la biblioteca científica estándar `scipy.optimize` (en concreto, el método Quasi-Newton BFGS y el método de programación cuadrática sucesiva SLSQP). Finalmente, se introduce el modelado con `CVXPY` para optimización convexa disciplinada (DCP), destacando la facilidad de planteamiento y la obtención directa de las variables duales (multiplicadores de Lagrange) para verificar las condiciones de optimalidad de Karush-Kuhn-Tucker (KKT).
- **Fichero Fuente:** `lab2_opt_no_lineal.qmd`

Laboratorio 3: Análisis de Convexidad y Modelado con CVXPY

- **Descripción:** Este laboratorio profundiza en el análisis práctico de la convexidad y en la optimización utilizando Python. Se estudian dos vertientes principales: la verificación de la convexidad a través del análisis simbólico y numérico de los autovalores de la Hessiana mediante

SymPy y NumPy, y el modelado y resolución de problemas bajo las reglas de la Programación Convexa Disciplinada (DCP) con CVXPY. A lo largo del laboratorio, se analizan las causas del error `DCPError` ante formulaciones no válidas, se estudia cómo reformular restricciones complejas (como raíces y productos cruzados) utilizando funciones atómicas como la media geométrica, y se obtienen e interpretan los multiplicadores de Lagrange (valores duales) para el análisis de sensibilidad de las restricciones.

- **Fichero Fuente:** `lab3_analisis_convexo.qmd`

Laboratorio 4: Condiciones de Optimalidad y KKT en Python

- **Descripción:** Esta sesión aborda el análisis computacional de las condiciones de optimalidad de Karush-Kuhn-Tucker (KKT) en problemas con y sin restricciones. El alumno implementa soluciones analíticas simbólicas en `SymPy` para evaluar la regularidad de las restricciones (cualificación LICQ mediante el rango de la matriz jacobiana activa), formula la ecuación de Fritz-John en casos singulares y programa la resolución combinatoria del sistema KKT derivado de las condiciones de holgura complementaria. Finalmente, contrasta los resultados analíticos con el modelado numérico en `CVXPY` y la extracción explícita de multiplicadores duales de Lagrange.
- **Fichero Fuente:** `lab4_condiciones_optimalidad.qmd`

Laboratorio 5: Dualidad KKT y Programación Cuadrática en Python

- **Descripción:** En este laboratorio se estudia la formulación y resolución práctica de la dualidad en Programación Lineal (LP) y Cuadrática (QP). El estudiante aprende a recuperar e interpretar los multiplicadores duales (precios sombra) mediante el atributo `.dual_value` en `CVXPY` para realizar análisis de sensibilidad sobre los términos independientes de las restricciones. Asimismo, implementa en Python el algoritmo del Símplex de Wolfe programando la regla de entrada restringida para garantizar la holgura complementaria y visualiza la trayectoria de convergencia sobre la región factible poliédrica.
- **Fichero Fuente:** `lab5_duales_lp_qp.qmd`

Laboratorio 6: Grafos y Árboles Soporte de Mínimo Peso en Python

- **Descripción:** Este laboratorio introduce la construcción, manipulación y análisis computacional de redes valoradas utilizando la biblioteca `NetworkX` en combinación con `NumPy` y `Matplotlib`. Se implementa la verificación numérica del *Handshaking Lemma* y el cálculo de métricas topológicas (grados, densidad, distancias geodésicas). El alumno programa desde cero la estructura de datos *Union-Find* (con compresión de caminos y unión por rango) para implementar los algoritmos de Kruskal y Prim, finalizando con la aplicación del clustering basado en MST (*Single-linkage*) para la detección de patrones no convexos en ciencia de datos.
- **Fichero Fuente:** `lab6_grafos_arboles.qmd`

Laboratorio 7: Problemas de Camino Mínimo en Python

- **Descripción:** En este laboratorio se aborda la formulación, implementación y análisis algorítmico de problemas de camino mínimo sobre redes valoradas en Python. Se emplean las librerías `NetworkX`, `NumPy` y `CVXPY`. El estudiante programa desde cero los algoritmos de Dijkstra y Bellman-Ford con criterio de detección de circuitos de coste negativo en tiempo $O(|V| \cdot |U|)$. Asimismo, se formula el problema de flujo de coste mínimo primal-dual en `CVXPY` extrayendo los potenciales nodales duales y se programa el algoritmo de Floyd-Warshall en `NumPy` para obtener la matriz de distancias absolutas entre todos los pares de nodos.
- **Fichero Fuente:** `lab7_caminos_minimos.qmd`

Laboratorio 8: Flujos en Redes en Python

- **Descripción:** Este laboratorio se centra en la formulación, resolución y análisis computacional de problemas de flujo en redes. Utilizando `NetworkX`, el alumno modela redes de flujo dirigidas y resuelve el problema de Flujo Máximo mediante `nx.maximum_flow`, verificando el Teorema del Flujo Máximo - Corte Mínimo con `nx.minimum_cut`. Asimismo, programa problemas de Flujo a Coste Mínimo mediante `nx.min_cost_flow` y `nx.network_simplex`. Finalmente, formula el problema general de transbordo en `CVXPY` con balances nodales y cotas de capacidad, extrayendo las variables duales nodales para validar la interpretación económica de los potenciales de red.
- **Fichero Fuente:** `lab8_flujos_redes.qmd`

Laboratorio 9: Emparejamientos en Redes en Python

- **Descripción:** Esta sesión práctica aborda los problemas de emparejamiento (*matching*) y asignación en redes mediante `NetworkX` y `SciPy`. El estudiante resuelve problemas de emparejamiento bipartito de máximo cardinal (`nx.bipartite.maximum_matching`) y el problema de asignación lineal óptima con `scipy.optimize.linear_sum_assignment`. Para grafos generales no bipartitos con ciclos impares, aplica el algoritmo de la Flor de Edmonds (*Blossom*) con `nx.max_weight_matching`. Adicionalmente, calcula recubrimientos de aristas mínimos (`nx.min_edge_cover`) comprobando el Teorema de Dualidad de Gallai ($|K_0| + |F_0| = |V|$), y formula el modelo de Programación Lineal Entera (PLE) en `scipy.optimize.milp` analizando la brecha de integrabilidad.
- **Fichero Fuente:** `lab9_emparejamientos.qmd`

Laboratorio 10: Rutas Eulerianas, Hamiltonianas, TSP y VRP en Python

- **Descripción:** Este laboratorio final está dedicado a la implementación computacional de problemas clásicos de rutas en redes empleando `NetworkX`, `SciPy`, `PuLP` y `Matplotlib`. El alumno programa el algoritmo constructivo de circuitos eulerianos y resuelve el Problema del Cartero Chino (CPP) mediante emparejamiento de peso mínimo de nodos impares. Asimismo, formula y resuelve el Problema del Viajante de Comercio (TSP) como un programa lineal entero mediante el modelo compacto de Miller-Tucker-Zemlin (MTZ) en `PuLP`. Desarrolla

desde cero heurísticas constructivas (Vecino más Cercano) y de búsqueda local (operador 2-Opt), evalúa el algoritmo de aproximación de Christofides (garantía de cota 1.5) y modela un problema básico de Rutas de Vehículos con Capacidad (CVRP) con flota homogénea.

- **Fichero Fuente:** `lab10_rutas.qmd`